

Day 1

Python for AI Automation

Foundation for UiPath Developers

Insurance Domain Hands-on Lab



Facilitator Introduction



Surendra Panpaliya, DTM

Global AI Architect | Enterprise AI Consultant
Founder & CEO – GKTCS Innovations

Generative AI

Agentic AI

Cloud AI

MLOps

Data Engineering

Enterprise AI
Systems

26+ years of experience delivering large-scale technology transformation programs across AI, Generative AI, Agentic AI, and Cloud Computing.

26+ years experience

35,000+ professionals
trained

1000+ corporate
programs

300+ global enterprises

Training Promise

You will build a working claim processor
— step by step.



Today's Journey

Four hours. Many small wins.

**Learn Python by automating
one insurance claim workflow.**

**1
Basics**

**2
Files**

**3
API**

**4
Project**

Why Python for UiPath?

RPA plus code becomes AI-ready.

UiPath moves work. Python adds intelligence, data and APIs.



Robot



Python



AI



APIs

Business Scenario

Insurance claim validation journey.

**A claim arrives. We verify
policy, premium and coverage.**



Claim



Policy



Premium



Decision

Day 1 Claim Flow

The business process behind the code.



One clean path. Many real-world exceptions.

Execution Model

Workflow thinking meets script thinking.

**Python runs top to bottom.
Functions make it reusable.**



Run



Step



Reuse



Test

Live Demo: First Script

Start small. Run immediately.

```
print("Claim Automation Started")  
    claim_id = "CLM1001"  
    claim_amount = 75000  
print(claim_id, claim_amount)
```

**Every automation
starts with clean
inputs.**

Variables

Names for business values.

A variable should tell the business story clearly.

Amount

ID
Claim ID


Policy


Status

Lab: Basic Variables

Create claim fields and print them.

1. Create policy number

2. Add customer name

3. Set claim type

4. Set claim amount

5. Print clean output



Expected Output

Readable claim details

Data Types

The small thing that breaks big bots.

Text, numbers and dates must be handled deliberately.

ABC
String

123
Integer



Date



Boolean

Functions

Reusable blocks of automation logic.

A function is a mini-workflow with a name.



Check



Calculate



Decide



Return

Validator Function

One rule. Reuse many times.

```
def check_claim_amount(claim, coverage):  
    if claim <= coverage:  
        return True  
    return False
```

**Small functions make
large automations
reliable.**

Lab: Reusable Validators

Policy, premium and amount checks.

1. Check policy status

2. Check premium status

3. Check claim amount

4. Combine all checks

5. Print final decision



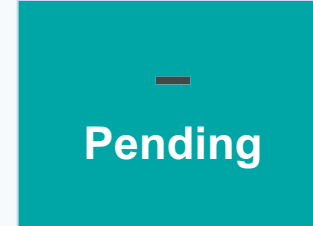
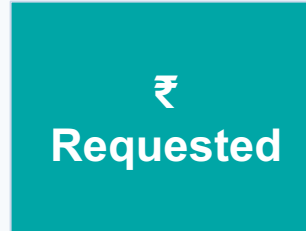
Expected Output

Claim eligibility result

Mini Assignment

Calculate pending claim amount.

**Requested amount minus
approved amount equals
pending amount.**



Modules Thinking

From notebook to reusable utility.

Today one file. Tomorrow a shared automation library.



Module



Utility



Reuse



Toolkit

Dictionaries

Python's business object format.

A dictionary stores insurance data as key-value pairs.



Key



Value



Record



JSON

Policy as Dictionary

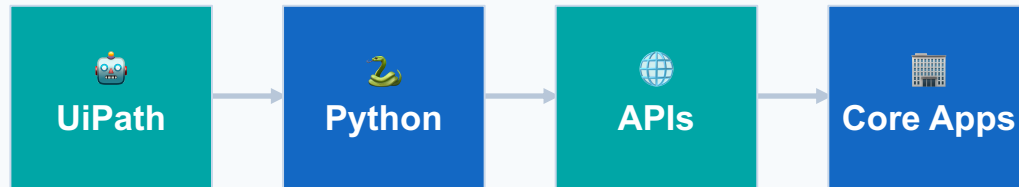
Structured data in memory.

```
policy = {  
  "policy_number": "POL1001",  
  "coverage_amount": 500000,  
  "status": "Active"  
}
```

**Dictionary today. JSON
tomorrow. API payload
later.**

Insurance Data Model

How records connect.



UiPath orchestrates. Python processes. APIs connect.

Files in Automation

Most bots begin with files.

Excel, CSV and JSON are automation highways.



Excel



CSV



JSON



Output

Create Sample Data

Build training files inside notebook.

1. Create data folder

2. Create policy records

3. Save Excel file

4. Save CSV file

5. Save claim JSON



Expected Output

Ready lab dataset

Read Excel with Pandas

Policy master becomes DataFrame.

A DataFrame is a table you can filter and validate.



Excel



Pandas



Filter



Validate

Lab: Policy Master

Read and inspect Excel data.

1. Load Excel file

2. Display records

3. Check columns

4. Filter active policies

5. Find pending premium



Expected Output

Clean policy view

Filter Policies

Business rules as code.

```
active = df[df["policy_status"] == "Active"]  
paid = df[df["premium_status"] == "Paid"]
```

**Filtering turns data into
decisions.**

Mini Lab: Good Policies

Filter active, paid and high coverage.

1. Select active policies

2. Select paid premium

3. Check coverage > 400000

4. Combine filters

5. Display final table



Expected Output

Eligible policy list

JSON Claim Data

API-style input for automation.

JSON is how modern systems exchange business data.

A teal square containing a white curly brace icon representing JSON.

JSON

A blue square containing a white envelope icon with a red arrow pointing into it, representing input.

Input

A teal square containing a white chain link icon representing an API.

API

A blue square containing a white robot head icon representing a bot.

Bot

Lab: Read Claim JSON

Parse one submitted claim.

1. Open JSON file

2. Load with `json.load`

3. Print claim ID

4. Print policy number

5. Print claim amount



Expected Output

Claim summary fields

Claim Summary

Readable output for business users.

```
summary = f"Claim {claim_id} submitted by {customer} for  
           {claim_type}."  
print(summary)
```

**Automation output
should be human-
readable.**

Exception Handling

Production will test your optimism.

Good automation expects errors and responds gracefully.



Error



Handle



Log



Recover

Try / Except

Protect the automation flow.

```
try:  
    data = json.load(file)  
    print(data["claim_id"])  
except KeyError:  
    print("Missing field")
```

Fail clearly. Recover safely.

Lab: Error Handling

Make failures friendly.

1. Rename input file

2. Run the code

3. Catch file error

4. Remove claim field

5. Catch missing key



Expected Output

Controlled error messages

REST API Basics

Python can call enterprise systems.

**APIs let automation verify
information in real time.**

GET
Fetch

POST
Create

PUT
Update


Secure

Mock Policy API

Practice without external dependency.

A mock API behaves like a real system for learning.



Mock



Policy



Premium



Response

Lab: Verify Policy

Call mock API and inspect response.

1. Pass policy number

2. Receive policy details

3. Check active status

4. Check premium paid

5. Display result



Expected Output

Policy verification result

End-to-End Processor

Files, rules and output together.



One clean path. Many real-world exceptions.

Decision Rules

Keep rules explicit and auditable.

Rejected claims must explain why they failed.



Active



Paid



Coverage



Reason

Lab: Final Decision

Combine policy and claim validation.

1. Read claim data

2. Find matching policy

3. Check policy status

4. Check premium status

5. Check coverage limit



Expected Output

Approved or rejected

Output Design

Clean output powers next automation.

A useful output is structured, clear and traceable.



Claim



Status



Reason



JSON

Lab: Generate Output

Save final claim result as JSON.

1. Create output dictionary

2. Add claim fields

3. Add validation status

4. Add decision reason

5. Save JSON file



Expected Output

final_claim_output.json

Batch Claims

From one claim to many.

Real automation rarely stops at one record.

1
Single


CSV


Loop


Report

Challenge Lab

Process multiple claims from CSV.

1. Read claims CSV

2. Loop through records

3. Validate each claim

4. Append output row

5. Save report CSV

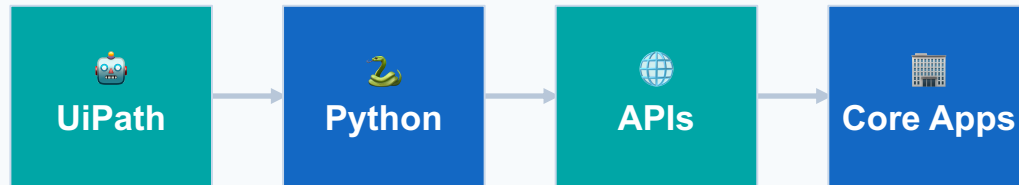


Expected Output

Batch claim report

UiPath + Python Architecture

Where each tool fits best.



UiPath orchestrates. Python processes. APIs connect.

Integration Patterns

Choose simple before complex.

**File handoff, command call,
API wrapper or queue.**



File



CLI



API



Queue

Where GenAI Helps

Tomorrow's intelligence layer.

**Summaries, explanations,
emails and document
understanding.**



Summary



Email



Reason



Docs

Where LangGraph Helps

When one rule becomes many agents.

**Claim intake, fraud, approval
and payment can become
agents.**



Intake



Fraud



Approval

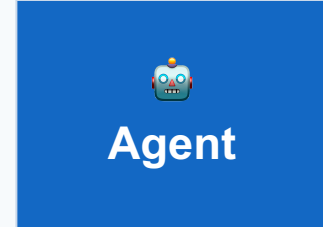
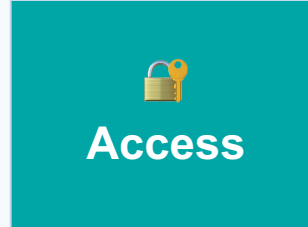
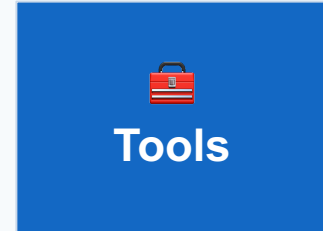
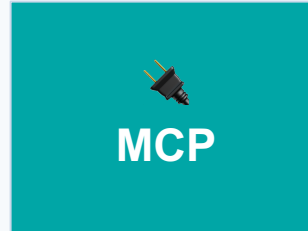


Payment

Where MCP Helps

Standard way for AI to use tools.

**MCP can expose files, APIs
and automation tools to
agents.**



Common Mistakes

Most bugs are small and loyal.

**Paths, spelling, indentation
and data types cause most
errors.**



Path



Spelling



Indent

123
Type

Quiz 1

Quick check before the next lab

Which format is closest to an API response?

A. Excel

B. JSON

C. PPT

D. Image

Answer: B. JSON

Quiz 2

Quick check before the next lab

What should a rejected claim include?

A. Only status

B. Rejection reason

C. Random log

D. Nothing

Answer: B. Rejection reason

Quiz 3

Quick check before the next lab

Which error comes from a missing dictionary field?

A. KeyError

B. FileNotFoundError

C. BotError

D. SlideError

Answer: A. KeyError

Final Assignment

Build the Insurance Document Processor.

**Read policy, read claim,
validate, decide and save
output.**

**1
Read**

**2
Validate**

**3
Decide**

**4
Save**

Solution Checklist

Use this before submitting.

**Correct inputs, clear functions,
handled errors, clean output.**



Inputs



Functions



Errors



Output

Discussion

Apply it to your process.

**Which claim step in your work
needs Python support?**



Discuss



Think



Apply



Share

Day 1 Recap

Python foundation is ready. Tomorrow we add Generative AI.



Thank You

Questions, lab review and next steps

From RPA Developer to AI Automation Engineer





Thank You

Surendra Panpaliya, DTM
Founder & CEO,
GKTCS Innovations

Empowering leaders to make smarter cloud, AI and digital transformation decisions.

Contact Information

-  **Email**
surendra@gktcs.com
-  **LinkedIn**
linkedin.com/in/surendrarp
-  **Website**
www.gktcs.com
-  **Phone**
+91 9975072320